

Table of contents

Série 5 : Vérification de programmes avec Why3	1
Méthodes	1
Spécification formelle avec préconditions et postconditions	1
Invariants de boucle	1
Prédicats auxiliaires	2
Propriétés des tableaux	2
Preuve de correction partielle (Hoare logic)	2
Rappel	2
Exemples Illustratifs	3
Exemple 1 : Spécification formelle avec préconditions et postconditions	3
Exemple 2 : Invariants de boucle	3
Exemple 3 : Propriétés des tableaux et prédicats auxiliaires	4
Synthèse	4

Série 5 : Vérification de programmes avec Why3

Méthodes

Spécification formelle avec préconditions et postconditions

Description : Dans la vérification de programmes, on spécifie le comportement attendu à l'aide d'assertions logiques.

- **Précondition** : formule devant être vraie avant l'exécution du programme (hypothèses sur les entrées).
- **Postcondition** : formule devant être vraie après l'exécution (garanties sur les sorties). Ces conditions sont exprimées en logique du premier ordre, souvent avec des quantificateurs sur les indices et valeurs des tableaux.

Invariants de boucle

Description : Un **invariant de boucle** est une formule logique qui reste vraie avant et après chaque itération d'une boucle. Il permet de raisonner sur la correction partielle et la terminaison. Pour une boucle **while**, l'invariant doit être :

- vrai avant la première itération,
- préservé par le corps de la boucle (si l'invariant est vrai avant l'itération et la condition de garde vraie, alors il reste vrai après l'itération),

- suffisamment fort pour impliquer la postcondition lorsque la condition de boucle devient fausse.

Prédicats auxiliaires

Description : Pour alléger les spécifications, on définit des **prédicats** (fonctions booléennes) qui encapsulent des propriétés réutilisables (ex. : permutation, tri). Ces prédicats peuvent être définis dans Why3 et utilisés dans les invariants, préconditions et postconditions.

Propriétés des tableaux

Description : Les algorithmes sur tableaux nécessitent d'exprimer des propriétés globales :

- **Tri** : ordre croissant ou décroissant des éléments.
- **Permutation** : conservation multie ensembliste des éléments (mêmes occurrences, ordre éventuellement différent).
- **Sous-tableau** : propriétés sur des segments d'indices.

Preuve de correction partielle (Hoare logic)

Description : La **logique de Hoare** permet de décomposer la preuve de correction d'un programme en vérifiant des triplets $\{P\} \ C \ \{Q\}$. Pour les boucles, on utilise un invariant. Why3 automatise la génération des obligations de preuve (verification conditions) à partir du code annoté.

Rappel

- **Lois de De Morgan** :
 $\neg(A \wedge B) \equiv \neg A \vee \neg B$
 $\neg(A \vee B) \equiv \neg A \wedge \neg B$
- **Élimination de l'implication** :
 $A \Rightarrow B \equiv \neg A \vee B$
- **Distributivité** :
 $A \wedge (B \vee C) \equiv (A \wedge B) \vee (A \wedge C)$
 $A \vee (B \wedge C) \equiv (A \vee B) \wedge (A \vee C)$
- **Quantificateurs** :
 $\forall i, P(i)$: pour tout i , $P(i)$ est vrai.
 $\exists i, P(i)$: il existe i tel que $P(i)$ est vrai.
 $\neg \forall i, P(i) \equiv \exists i, \neg P(i)$
 $\neg \exists i, P(i) \equiv \forall i, \neg P(i)$

- **Triplet de Hoare :**

$\{P\} C \{Q\}$ signifie : si l'exécution de C commence dans un état satisfaisant P , et si C termine, alors l'état final satisfait Q .

Exemples Illustratifs

Exemple 1 : Spécification formelle avec préconditions et postconditions

Contexte : Recherche binaire dans un tableau trié.

Application :

Le pseudocode de la recherche binaire est donné. La **précondition** exige que le tableau d'entrée soit trié :

$$\forall i, j \in \{0, \dots, n-1\}, i \leq j \Rightarrow a[i] \leq a[j]$$

La **postcondition** précise le résultat **res** :

$$(res \in \{0, \dots, n-1\} \wedge a[res] = elem) \vee (res = -1 \wedge \forall i \in \{0, \dots, n-1\}, a[i] \neq elem)$$

Cela garantit que si l'élément est présent, **res** est un indice valide où il se trouve, sinon **res** vaut -1 et l'élément est absent.

Exemple 2 : Invariants de boucle

Contexte : Recherche binaire, boucle **while**.

Application :

L'invariant proposé est :

$$I_1 = 0 \leq l \wedge u \leq n-1 \wedge \forall i \in \{0, \dots, n-1\}, a[i] = elem \Rightarrow l \leq i \leq u$$

où l et u sont les bornes courantes de la zone de recherche.

Interprétation :

- Les bornes l et u restent dans les limites du tableau (ou aux bords).
- Si l'élément **elem** est présent dans le tableau, alors il se trouve nécessairement dans l'intervalle $[l, u]$.

À chaque itération, on calcule le milieu m . Selon la comparaison entre $a[m]$ et $elem$, on met à jour l ou u de manière à préserver l'invariant. Quand la boucle se termine ($l > u$), on peut conclure la postcondition.

Exemple 3 : Propriétés des tableaux et prédicats auxiliaires

Contexte : Tri à bulles.

Application :

La **postcondition** doit exprimer que le tableau de sortie est trié et est une permutation du tableau d'entrée. On définit un prédicat auxiliaire `permut_all` (par exemple) :

```
predicate permut_all (a1: array int) (a2: array int) =  
  length a1 = length a2 /\  
  forall v: int. occurrences v a1 = occurrences v a2
```

La postcondition s'écrit alors :

$$\text{sorted}(a) \wedge \text{permut_all}(a, a_{\text{initial}})$$

où `sorted` est un prédicat exprimant l'ordre croissant.

Pour les **invariants des boucles imbriquées** :

- **Boucle externe** (i) : les éléments de $a[i+1 \dots n-1]$ sont triés et sont les plus grands ; le sous-tableau $a[0 \dots i]$ est une permutation du sous-tableau initial correspondant.
- **Boucle interne** (j) : le sous-tableau $a[0 \dots j-1]$ est trié et tous ses éléments sont inférieurs ou égaux à $a[j]$ (pour la version ascendante).

Ces invariants permettent de prouver que chaque passe place le plus grand élément restant à sa position finale.

Synthèse

Cette série d'exercices illustre les méthodes fondamentales de la vérification formelle de programmes impératifs avec Why3. Les points clés sont :

1. **Spécification rigoureuse** via préconditions et postconditions en logique du premier ordre.
2. **Utilisation d'invariants de boucle** pour capturer l'état intermédiaire et guider la preuve.

3. **Définition de prédicats auxiliaires** pour abstraire des propriétés complexes (tri, permutation).
4. **Application à des algorithmes classiques** : recherche binaire, tri à bulles, tri par sélection.

Ces méthodes permettent de démontrer la **correction partielle** (si le programme termine, il satisfait sa spécification) et, avec des arguments supplémentaires (variants), la **terminaison**.